U!REKA
EUROPEAN UNIVERSITY

Departamento de Engenharia de Electrónica e Telecomunicações e de Computadores

Play4Change: Adaptive Learning Powered by AI

A51620 : Radesh Ilesh Gamanbhai Govind (A51620@alunos.isel.pt)

Relatório para a Unidade Curricular de Projecto e Seminário 2025/26
da Engenharia Informática

Professores : Professor Adjunto Prof. Nuno Datia
Professor Adjunto Prof. António Serrador

18 – April – 2026

Abstract

Play4Change is an adaptive learning platform that combines generative AI, gamification mechanics, and peer review to transform educational content into personalized daily challenges. The platform addresses two critical societal needs: sustainability education aligned with the United Nations Sustainable Development Goals, and digital literacy for the next generation of responsible citizens.

From a technical perspective, the system comprises a cross-platform mobile application built with Kotlin Multiplatform (KMP) and Decompose for iOS and Android, a Spring Boot and Kotlin backend running in Docker, and a TypeScript/React landing page with a hidden admin panel for content management. A shared Kotlin module enforces type-safety and code reuse across client and server. Content generation is powered by an AI pipeline combining the Mistral model with Retrieval-Augmented Generation (RAG) and vector search.

The architecture adheres to Domain-Driven Design, hexagonal architecture, and Clean Architecture principles, with an emphasis on sustainable and predictable costs, security through passwordless authentication, accessibility via caching, and future scalability with Kubernetes.

Keywords: Adaptive Learning Gamification Generative AI Kotlin Multiplatform Sustainability Clean Architecture

Resumo

Play4Change é uma plataforma de aprendizagem adaptativa que combina inteligência artificial generativa, mecânicas de gamificação e revisão por pares para transformar conteúdo educativo em desafios diários personalizados. A plataforma dirige-se a duas necessidades críticas da sociedade contemporânea: a educação para a sustentabilidade alinhada com os Objetivos de Desenvolvimento Sustentável das Nações Unidas, e a literacia digital.

Do ponto de vista técnico, o sistema é composto por uma aplicação móvel multiplataforma desenvolvida com Kotlin Multiplatform (KMP) e Decompose para iOS e Android, um backend baseado em Spring Boot e Kotlin executado em Docker, e uma página de destino com painel de administração construída em TypeScript e React. Um módulo partilhado em Kotlin assegura type-safety e reutilização de lógica entre cliente e servidor. A geração de conteúdo é realizada através de um pipeline de IA que combina o modelo Mistral com Retrieval-Augmented Generation (RAG) e pesquisa vetorial.

A arquitetura segue princípios de Design Orientado a Domínio, arquitetura hexagonal e Clean Architecture, com ênfase em custos previsíveis, segurança (autenticação sem palavra-passe), acessibilidade e preparação para escalonamento futuro com Kubernetes.

Palavras-chave: Aprendizagem Adaptativa Gamificação Inteligência Artificial Generativa Kotlin Multiplatform Sustentabilidade Clean Architecture

Contents

Chapter 1

Introduction

1.1 Context and Motivation

The world simultaneously faces an education crisis and an engagement crisis. Sustainability topics — climate action, responsible consumption, digital citizenship — are recognized as urgent priorities by the United Nations through the 2030 Agenda for Sustainable Development Goals (SDGs) [**unsdgs2015**]. Yet traditional educational methods fail to convert knowledge into lasting behavioral change. Passive lectures, static documents, and one-size-fits-all curricula produce low retention and even lower motivation in learners who are bombarded with information but rarely empowered to act on it.

Parallel to this, digital literacy has become a fundamental skill in a world where misinformation spreads rapidly and critical thinking about technology is essential. Educators need accessible tools to build and deliver quality learning experiences without requiring deep technical expertise. Learners need experiences that fit their pace, respect their time, and reward consistent effort.

These two challenges share a common root: the gap between content and engagement. Existing platforms either focus on content delivery without personalization, or on engagement mechanics without educational rigor. Play4Change is designed to close that gap.

1.2 Problem Statement

Current e-learning platforms suffer from well-documented limitations:

- **Low engagement**: Completion rates for Massive Open Online Courses (MOOCs) rarely exceed 10% [**jordan2014initial**].
- **Lack of personalization**: Courses are delivered at a fixed pace regardless of the learner's prior knowledge or performance.
- **Content bottleneck**: Educators must manually author every quiz, challenge, and task, limiting scalability.
- **Weak feedback loops**: Learners receive little indication of their progress relative to their goals or peers.

- **High infrastructure cost**: Many platforms require expensive cloud resources that scale poorly with the user base.

From a systems engineering perspective, there is also a lack of rigorously architected platforms in the educational technology space. Most edtech solutions are built as monolithic applications with tight coupling, making them difficult to maintain, test, and scale.

1.3 Objectives

Play4Change addresses these problems by pursuing the following primary objectives:

1. Design and implement a **cross-platform mobile application** (iOS and Android) using Kotlin Multiplatform (KMP) with Decompose for lifecycle-aware navigation.
2. Build a **scalable, domain-driven backend** capable of delivering personalized daily challenges at predictable cost.
3. Integrate **generative AI** (Mistral LLM) with Retrieval-Augmented Generation (RAG) and vector search for automated, high-quality content creation from educator-supplied materials.
4. Apply **clean architecture, hexagonal architecture, and Domain-Driven Design** (DDD) principles consistently across all system layers.
5. Ensure the system prioritizes **security** (passwordless authentication, minimal sensitive data retention), **accessibility** (caching, low-bandwidth friendliness), and **future scalability** (Kubernetes-ready infrastructure, CDN-friendly design).
6. Provide a **curated topic ecosystem** aligned with UN Sustainable Development Goals and digital literacy curricula.

1.4 Supervisors

This project is supervised by:

- **Prof. Nuno Datia** — ISEL, Engenharia Informática
- **Prof. António Serrador** — ISEL, Engenharia Informática
- **Prof. Michel Vorenhout** — Ureka / Sustainability domain co-supervisor

1.5 Report Structure

The remainder of this report is organized as follows:

Chapter ?? Background and State of the Art — reviews the relevant literature on e-learning, gamification, adaptive learning, AI-generated education, sustainability curricula, and the architectural patterns employed in the system.

Chapter ?? Play4Change Platform — describes the product vision, core features, user roles, and the four-step learning journey.

Chapter ?? System Architecture — presents the technical architecture, technology stack, component breakdown, and the engineering principles that govern design decisions.

Chapter ?? Implementation — details the current development state, testing strategy, CI/CD pipeline, and known challenges.

Chapter ?? Conclusion and Future Work — summarizes the contributions, reflects on the project status, and outlines planned future work.

Chapter 2

Background and State of the Art

2.1 E-Learning Platforms

The e-learning industry has grown substantially over the past decade. Platforms such as Coursera, edX, Udemy, and Khan Academy have demonstrated that digital education can reach millions of learners globally. However, their core model — video lectures supplemented by multiple-choice assessments — has been repeatedly criticized for low engagement and poor knowledge retention [**jordan2014initial**].

Duolingo [**vesselinov2012duolingo**] is a notable exception: by applying gamification rigorously to language learning, it achieves significantly higher daily active usage than traditional platforms. Play4Change draws inspiration from this approach while extending it to sustainability and digital literacy topics and adding AI-driven content generation.

2.2 Gamification in Education

Gamification is defined by Deterding et al. [**deterding2011game**] as the use of game design elements in non-game contexts. In education, the most effective mechanics include:

- **Daily streaks**: reinforce consistent engagement through loss aversion.
- **Leaderboards**: provide social proof and healthy competition.
- **Achievement badges**: signal mastery and milestone completion.
- **Experience points (XP) and levels**: create a visible progression arc.
- **Peer review**: builds critical thinking and community knowledge.

Play4Change incorporates all of these mechanics. A key design principle is that gamification must serve learning, not replace it — badges are awarded for genuine comprehension, not mere participation.

2.3 Adaptive Learning

Adaptive learning systems adjust the difficulty, pacing, and content of lessons based on each learner's performance [**vanlehn2011relative**]. The goal is to keep learners in the flow state [**csikszentmihalyi1990flow**] — the optimal cognitive zone between boredom and anxiety.

Play4Change implements adaptive difficulty by tracking learner responses, time-to-answer, and streak continuity, then adjusting the complexity of generated challenges accordingly. The AI pipeline (see Section **??**) is responsible for generating challenges at the appropriate level.

2.4 Generative AI and Retrieval-Augmented Generation

Large Language Models (LLMs) have transformed content generation. However, raw LLM output suffers from hallucination and lack of grounding in specific source material. Retrieval-Augmented Generation (RAG) [**lewis2020retrieval**] addresses this by augmenting generation with a retrieval step: documents are embedded into a vector database and semantically similar passages are retrieved at inference time to ground the model's response.

Play4Change uses the **Mistral** LLM [**mistral2024**] for content generation, combined with a vector search index over educator-supplied PDFs and URLs. This ensures that generated quizzes, challenges, and explanations are faithful to the source material. The choice of Mistral is motivated by its strong performance-to-cost ratio and its availability via self-hosted and API-based deployments, supporting the platform's cost-sustainability goals.

2.5 Sustainability Education and the UN SDGs

The United Nations' 2030 Agenda defines 17 Sustainable Development Goals [**unsdgs2015**] covering poverty, health, education, climate action, and more. Education for Sustainable Development (ESD) is explicitly addressed in SDG 4 (Quality Education) and SDG 13 (Climate Action). Despite global commitments, integrating SDG content into daily learning routines remains a challenge.

Play4Change curates topics around the SDGs and digital citizenship, enabling educators to contribute materials that are automatically structured into learning paths aligned with these goals.

2.6 Passwordless Authentication

Traditional password-based authentication is a leading cause of security breaches: credential stuffing, phishing, and password reuse affect virtually every consumer platform. The FIDO2 standard [**fido2**] and related approaches (magic links, OTPs, biometric device authentication) eliminate shared secrets entirely, reducing attack surface while improving user experience.

Play4Change adopts passwordless authentication as a first principle, avoiding the storage of password hashes entirely. This reduces both security liability and compliance burden.

2.7 Cross-Platform Mobile Development

Three major approaches compete for cross-platform mobile development:

- **React Native** (Meta): JavaScript/TypeScript bridge to native components. Fast iteration but performance overhead.
- **Flutter** (Google): Dart with a custom rendering engine. Good performance but non-native UI and large binary sizes.
- **Kotlin Multiplatform (KMP)** [**kotlinmultiplatform2024**]: shares business logic and data layers across platforms while keeping UI native. No bridge overhead, full access to platform APIs.

Play4Change uses KMP because it allows the team to share domain logic, network code, serialization, and database queries across iOS and Android, while writing native in Jetpack Compose (Android) and SwiftUI (iOS). This maximizes code reuse without sacrificing the platform-specific user experience that users expect.

Decompose [**decompose2024**] is used for navigation and lifecycle management within KMP. It provides a component-based navigation model that works identically on both platforms, avoiding the need to re-implement navigation logic twice.

2.8 Backend Technologies

Spring Boot [**springboot2024**] with Kotlin is the chosen backend framework. Kotlin's null-safety, coroutine support, and seamless interoperability with the Java ecosystem make it an ideal fit. The backend is containerized with Docker for reproducible deployments.

Add specific Docker Compose / orchestration diagram once finalized.

2.9 Architectural Patterns

2.9.1 Domain-Driven Design

Domain-Driven Design (DDD) [**evans2003domain**] organizes software around the business domain rather than technical concerns. Key DDD building blocks used in Play4Change include aggregates, value objects, domain events, and bounded contexts. Each subsystem (topics, challenges, learner progress, gamification) is modeled as a separate bounded context with its own domain model.

2.9.2 Hexagonal Architecture

The hexagonal architecture [**cockburn2005hexagonal**] (also known as Ports and Adapters) separates the application core from its infrastructure dependencies. The core defines ports (interfaces), and adapters implement those ports for specific technologies (HTTP, database, message queue). This makes it trivial to swap infrastructure components without touching business logic, and enables thorough unit testing of the domain without any infrastructure.

2.9.3 Clean Architecture

Clean Architecture [**martin2017clean**] enforces a strict dependency rule: inner layers know nothing about outer layers. In Play4Change, the layering is: **Domain** → **Application** → **Infrastructure** → **Presentation**. This guarantees that domain logic is framework-agnostic and testable in isolation.

2.10 Related Work and Gap Analysis

Table **??** summarizes the key differentiators of Play4Change relative to existing platforms.

Table 2.1: Comparison of Play4Change with related platforms.

Feature	Duolingo	Coursera	Khan Academy	Play4Change
AI content generation	×	×	×	✓
Adaptive difficulty	✓	×	Partial	✓
Daily streak mechanics	✓	×	×	✓
Peer review tasks	×	Partial	×	✓
SDG-focused topics	×	Partial	Partial	✓
Passwordless auth	×	×	×	✓
KMP cross-platform	×	×	×	✓

The primary gap Play4Change fills is the combination of AI-powered content creation, adaptive gamified delivery, and rigorous software engineering (type-safe shared module, DDD, clean architecture) in a single platform focused on sustainability and digital literacy.

Chapter 3

Play4Change Platform

3.1 Vision and Value Proposition

Play4Change transforms passive knowledge into active engagement. The platform makes sustainability education and digital literacy accessible, habit-forming, and impactful — one daily challenge at a time. Its core value proposition rests on three pillars:

1. **Learning that creates real change**: content is curated and AI-enriched so that every challenge connects to real-world behavioral outcomes aligned with the UN SDGs.
2. **Minimum friction, maximum impact**: learners receive one challenge per day — bite-sized and achievable — reducing the activation energy required to engage consistently.
3. **A healthy system for providers**: the platform is designed so that its infrastructure costs scale predictably with the user base, preventing the platform from becoming an operational liability.

3.2 Core Features

3.2.1 AI-Generated Content

Topics are created by educators who upload PDF materials or provide URLs. The AI pipeline (Mistral + RAG) processes this content and automatically generates:

- Multiple-choice and open-ended **quizzes**;
- **Daily challenge** prompts (reflection, action-based, or knowledge tasks);
- **Difficulty variants** for adaptive delivery.

Educators review and curate the generated content through the admin panel before it is made available to learners.

3.2.2 Adaptive Learning

The platform adapts to each learner's pace and performance. After each challenge response, the system evaluates correctness, response time, and streak continuity to place the learner on a difficulty trajectory. Learners performing consistently well receive progressively harder challenges; those struggling receive scaffolded hints and easier variants.

3.2.3 Gamification and Peer Review

The gamification layer includes:

- **Daily streaks**: learners are notified if their streak is at risk, leveraging loss aversion.
- **Leaderboards**: scoped to a topic or global, updated daily.
- **Achievement badges**: awarded for milestones (first challenge, 7-day streak, topic completion, peer review contribution).
- **Peer review tasks**: learners evaluate open-ended responses from peers, developing critical thinking and community knowledge.

3.2.4 Sustainability Focus

All topics on the platform are curated around the UN Sustainable Development Goals and digital citizenship. Each topic is tagged to one or more SDGs, allowing learners to explore specific goals (e.g., SDG 13 Climate Action, SDG 4 Quality Education) and track their engagement across the SDG framework.

3.3 User Roles

Play4Change defines three primary user roles:

Administrator (Educator) Can create and manage topics, upload learning materials, review AI-generated content, and publish challenges. Administrators access the platform via the hidden admin panel on the landing page.

Learner Can browse and enroll in topics, receive daily challenges, submit responses, participate in peer review, and track their progress and achievements.

System (AI Agent) The internal AI component that processes educator materials, generates challenge content, and adapts difficulty — not a human user but an active system actor.

3.4 The Four-Step Learning Journey

Play4Change bridges the gap between educational content and real behavioral change through a four-step process:

Step 1 — Educators Create Topics Administrators upload PDF materials or provide URLs pointing to relevant resources. The AI pipeline processes the content, generates a structured learning path with daily challenges, and presents it for educator review and approval.

Step 2 — Learners Enroll and Start Students discover topics through the mobile application, enroll in seconds, and begin receiving one challenge per day. The bite-sized format fits any schedule, requiring as little as five minutes of daily engagement.

Step 3 — Progress Through Challenges Each day brings a new challenge — quiz, reflection prompt, or peer task. The AI monitors performance and adapts difficulty to keep each learner in the flow state. Push notifications maintain streak momentum.

Step 4 — Earn Achievements and Grow Learners complete streaks, earn badges, and climb leaderboards. Community peer reviews build deeper understanding and collaborative knowledge. Progress is visualized through the learner's profile, showing SDG engagement and mastery levels per topic.

3.5 Landing Page and Administration

The public-facing landing page is built with TypeScript and React. It serves as the primary marketing and onboarding surface for the platform. A hidden administration route — accessible only to authenticated administrators — provides the content management interface without exposing it to the general public.

Add screenshot of landing page and admin panel once UI is finalized.

3.6 Platform Constraints and Design Philosophy

Play4Change is designed around the following non-negotiable properties:

- **Sustainable, predictable costs**: infrastructure is sized for actual load, not theoretical peaks. Caching and CDN reduce origin server pressure significantly.
- **Accessibility**: the platform must be usable on low-bandwidth connections. Challenge payloads are small; heavy content (videos, large images) is served via CDN.
- **Security by minimization**: the system avoids storing sensitive information wherever possible. Passwordless authentication eliminates the password database entirely.
- **Growth without penalty**: the architecture is designed so that a 10× increase in users does not require a 10× increase in cost or engineering effort. Kubernetes orchestration (planned) and horizontal scaling are first-class citizens.

Chapter 4

System Architecture

4.1 Overview

The Play4Change system is composed of four primary components that interact through well-defined interfaces (Figure **??**):

1. **Mobile Client** — Kotlin Multiplatform (KMP) application for iOS and Android.
2. **Backend Server** — Spring Boot application running in Docker.
3. **Shared Kotlin Module** — type-safe contracts and domain models shared between client and server.
4. **Landing Page & Admin Panel** — TypeScript/React web application.

Insert system overview architecture diagram here (Figure **??**) once finalized.

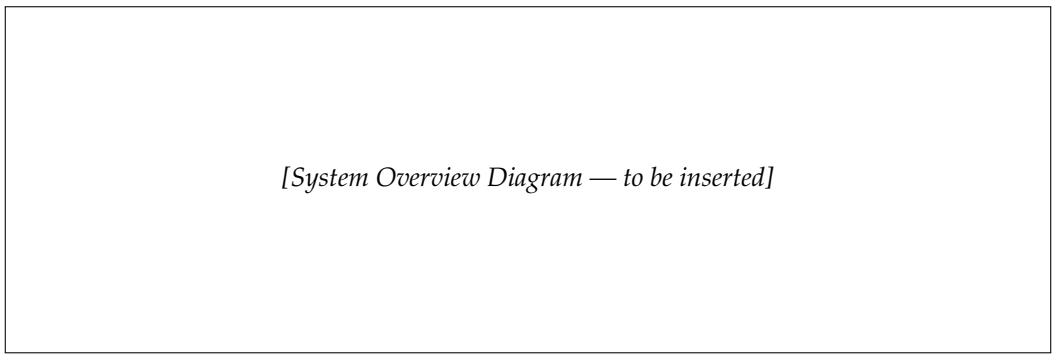

Figure 4.1: High-level system overview of Play4Change.

4.2 Architectural Principles

Every component in the system follows the same three-layer architectural philosophy:

4.2.1 Domain-Driven Design (DDD)

The domain is decomposed into the following bounded contexts:

- **Identity**: user accounts, authentication tokens, sessions.
- **Content**: topics, learning materials, AI-generated challenges.
- **Learning**: enrollments, daily challenge delivery, adaptive difficulty.
- **Gamification**: streaks, badges, leaderboards, XP.
- **Peer Review**: task assignment, submission, and evaluation workflow.

Each bounded context has its own aggregate roots, value objects, and domain events. Communication across bounded contexts is event-driven where possible, ensuring loose coupling.

4.2.2 Hexagonal Architecture (Ports and Adapters)

Each bounded context exposes input ports (use case interfaces) and output ports (repository and external service interfaces). Adapters implement the output ports for concrete technologies: `JPA` for relational data, a vector database client for embeddings, an HTTP adapter for the Mistral API, and SMTP/push notification adapters for delivery.

This structure means the domain and application layers contain zero framework imports. All Spring annotations, JPA annotations, and HTTP client code live exclusively in the infrastructure layer.

4.2.3 Clean Architecture Layering

The dependency rule is strictly enforced:

1. **Domain Layer**: entities, value objects, domain events, domain services. No external dependencies.
2. **Application Layer**: use cases, command/query handlers, application services. Depends only on domain.
3. **Infrastructure Layer**: persistence adapters (JPA repositories), external API clients (Mistral, push), messaging. Depends on application ports.
4. **Presentation Layer**: REST controllers, serialization (Kotlinx.serialization), request/response DTOs. Depends on application use cases.

4.3 Shared Kotlin Module

One of the key engineering decisions in Play4Change is the **shared Kotlin module** — a common library compiled for both the JVM (server) and Kotlin/Native or Kotlin/JS (clients). It contains:

- **API contracts**: request/response data classes with `kotlinx.serialization` annotations, ensuring the client and server can never fall out of sync.
- **Domain value objects**: identifiers, enumerations, and constraints shared across the system boundary.
- **Validation logic**: field-level constraints (e.g., minimum password length — even if no password is stored, magic link token constraints) that execute identically on client and server.

This module is the primary mechanism for **type safety across the system boundary**. If the backend changes an API response shape, the shared module update forces the client to adapt at compile time, eliminating an entire class of runtime integration bugs.

4.4 Mobile Client — Kotlin Multiplatform

4.4.1 KMP Architecture

The mobile client is structured as a KMP project with the following source sets:

- `commonMain`: business logic, navigation components, network clients, data repositories, view models.
- `androidMain`: Android-specific entry points, Jetpack Compose UI, platform services (notifications, biometrics).
- `iosMain`: iOS-specific entry points, SwiftUI integration, platform services.

Add KMP module dependency diagram once finalized.

4.4.2 Navigation with Decompose

Navigation and lifecycle management are handled by **Decompose** [**decompose2024**]. Each screen is a component with its own lifecycle, child stack, and back-stack. The `RootComponent` manages the top-level navigation graph; child components manage sub-graphs (e.g., the onboarding flow, the topic browser, the daily challenge flow).

Decompose's component model ensures that:

- Navigation logic lives in `commonMain` and is platform-independent.
- ViewModels are lifecycle-aware without relying on Android's ViewModel class.
- Deep linking and state restoration work consistently across both platforms.

4.4.3 Data Layer

The client data layer uses:

- **Ktor** for HTTP communication with the backend.
- **SQLDelight** for local persistence (challenge cache, offline access).
- **Kotlinx.serialization** for JSON marshaling (shared with the server via the shared module).

4.5 Backend — Spring Boot with Kotlin

4.5.1 Technology Stack

Table 4.1: Backend technology stack.

Component	Technology
Language	Kotlin (JVM 21)
Framework	Spring Boot 3.x
Persistence	Spring Data JPA + PostgreSQL
Vector Search	
Messaging	
Caching	Redis
Containerization	Docker / Docker Compose
Auth	Passwordless (magic link / OTP)

4.5.2 API Design

The backend exposes a RESTful JSON API consumed by both the mobile client and the admin panel. API versioning is handled via URL prefixes (`/api/v1/...`). All endpoints are documented with OpenAPI (Springdoc).

Link to OpenAPI specification in Appendix once generated.

4.5.3 Docker and Containerization

All backend services are containerized. A `docker-compose.yml` file orchestrates the full local development environment: the Spring Boot application, PostgreSQL, Redis, and the vector database. This ensures environment parity between development, CI, and production.

4.6 AI Content Pipeline

The AI content pipeline is responsible for transforming raw educator materials into structured, graded challenges. It operates as follows:

1. **Ingestion**: an educator uploads a PDF or provides a URL. The document is chunked into overlapping passages.
2. **Embedding**: each passage is embedded using a sentence-level embedding model and stored in the vector database.
3. **Generation**: when content is needed, relevant passages are retrieved by semantic similarity (RAG [**lewis2020retriev**] and passed as context to the Mistral LLM [**mistral2024**], which generates challenges in a structured JSON schema.
4. **Validation**: generated challenges are schema-validated and flagged for educator review if confidence is below a threshold.
5. **Adaptation**: the difficulty of generated challenges is tagged so the adaptive engine can select the appropriate variant per learner.

Add AI pipeline diagram once architecture is finalized.

4.7 Security Design

4.7.1 Passwordless Authentication

Play4Change implements passwordless authentication using magic links and OTPs [**fido2**]. The authentication flow is:

1. Learner enters their email address.
2. The server generates a short-lived, single-use token and emails a magic link.
3. The learner clicks the link; the token is validated and exchanged for a session JWT.
4. On mobile, biometric device authentication (FaceID/fingerprint) can be used after initial email verification.

No password hashes are stored. The attack surface is reduced to session token security (short-lived JWTs with refresh rotation) and email account security.

4.7.2 Minimal Data Retention

In line with privacy-by-design principles, Play4Change minimizes the data it collects and retains:

- No password hashes, no payment data, no third-party tracking pixels.
- Challenge response data is retained for personalization purposes only.
- Users can request full data deletion (GDPR Article 17 compliance).

4.8 Infrastructure and Scalability

4.8.1 Caching Strategy

Redis is used as a multi-level cache:

- **Challenge cache**: daily challenges are pre-generated and cached per learner at midnight, so the delivery endpoint is a cache hit 99% of the time.
- **Leaderboard cache**: leaderboard rankings are recomputed periodically and served from cache, avoiding expensive real-time aggregation queries.
- **API response cache**: frequently accessed, rarely changing data (topic metadata, badge definitions) is cached with TTL-based invalidation.

4.8.2 CDN Integration

Static assets (landing page, images, large topic resources) are served via a CDN. This reduces origin server load, improves global latency, and enables offline-tolerant Progressive Web App behavior for the landing page.

Specify CDN provider (Cloudflare / AWS CloudFront / BunnyCDN) once decided.

4.8.3 Kubernetes Readiness

Although the initial deployment uses Docker Compose, the system is designed with Kubernetes in mind [**kubernetes2024**]:

- All services are stateless; state lives in PostgreSQL, Redis, and the vector database.
- Health check endpoints (`/actuator/health`) are exposed for liveness and readiness probes.
- Configuration is externalized via environment variables (12-factor app).
- Horizontal pod autoscaling (HPA) can be applied to the API and AI pipeline services independently.

4.9 CI/CD Pipeline

The project uses a CI/CD pipeline that enforces quality gates at every push:

1. **Lint and static analysis**: Detekt (Kotlin) and ESLint (TypeScript) run on every pull request.
2. **Unit and integration tests**: the full test suite runs in Docker, using the same database image as production.
3. **Build**: Docker images are built and tagged.

4. **Deploy**: on merge to `main`, the latest image is deployed to the staging environment.

Insert CI/CD pipeline diagram and specify platform (GitHub Actions / GitLab CI) once confirmed.

4.10 Testing Strategy

The test pyramid is followed throughout the codebase:

- **Unit tests**: domain logic and use cases are unit-tested in isolation (no infrastructure). Coverage target: $\geqslant$80% for domain and application layers.
- **Integration tests**: repository adapters are tested against a real PostgreSQL instance using Testcontainers.
- **End-to-end tests**: critical user journeys (enrollment, daily challenge submission, achievement award) are tested against the deployed staging environment.
- **KMP tests**: shared module logic is tested in both JVM and Kotlin/Native execution environments.

Chapter 5

Implementation and Current State

5.1 Development Status

Play4Change is currently under active development. Table **??** summarizes the implementation status of each major component as of the time of writing.

Table 5.1: Development status by component.

Component	Status	Notes
Shared Kotlin Module	In Progress	API contracts and domain value objects defined; validation logic pending.
Backend — Domain Layer	In Progress	Core aggregates modeled; domain events wired.
Backend — Application	In Progress	Primary use cases scaffolded.
Backend — Infrastructure	In Progress	JPA adapters for Identity and Content contexts; Redis caching integration pending.
Backend — REST API	In Progress	Auth and Topics endpoints implemented; Challenges and Gamification pending.
AI Pipeline	In Progress	Ingestion and embedding operational; adaptive difficulty scoring pending.
KMP Client — Android	In Progress	Onboarding and topic browser screens implemented.
KMP Client — iOS	In Progress	Shared components compiling; SwiftUI integration in progress.
Landing Page	In Progress	Marketing content and onboarding flow implemented; admin panel in progress.
CI/CD	Configured	Lint and build gates active; integration tests pending.

Update Table **??** as components reach completion.

5.2 Key Implementation Decisions

5.2.1 Type Safety via the Shared Module

The shared Kotlin module is the most impactful single engineering decision in the project. All HTTP request and response bodies are defined as `@Serializable` Kotlin data classes in `commonMain`. Both the Ktor HTTP client on the mobile side and the Spring Boot controller on the server side use these same classes. A change to an API response that is not reflected in the shared module produces a compile-time error, not a runtime crash.

5.2.2 RAG Pipeline Implementation

The RAG pipeline uses a two-stage approach:

1. **Offline indexing**: when a topic is created, the educator's materials are chunked (with overlap to preserve context across chunk boundaries), embedded, and stored in the vector database. This process is asynchronous and does not block the educator's workflow.
2. **Online generation**: when the scheduler triggers challenge generation for a topic, the most relevant chunks are retrieved by cosine similarity to a generation prompt, and passed as context to Mistral. The model is prompted to generate challenges in a strict JSON schema with fields for question, options, correct answer, difficulty level, and SDG tags.

5.2.3 Adaptive Difficulty Scoring

Detail the adaptive scoring algorithm once finalized. Planned approach: Elo-based rating per learner per topic, updated after each challenge submission.

5.2.4 Passwordless Flow

The authentication service generates a HMAC-SHA256 signed token containing the user's email and an expiry timestamp (15 minutes). The token is embedded in a magic link sent by email. On click, the server validates the signature and expiry, creates or retrieves the user account, issues a JWT (15-minute expiry) and a refresh token (7-day expiry, rotated on use). Refresh tokens are stored hashed in PostgreSQL. No plaintext sensitive data persists.

5.3 Encountered Challenges

5.3.1 KMP and iOS Interoperability

Integrating Decompose with SwiftUI required careful management of the Kotlin/Native memory model and Objective-C interoperability. Component callbacks and StateFlow collectors must be managed through Decompose's lifecycle to prevent memory leaks on iOS.

Document final integration pattern once stabilized.

5.3.2 Mistral Prompt Engineering

Generating consistently structured JSON from an LLM requires careful prompt engineering. Early iterations produced well-formed text challenges but inconsistent JSON schemas. The solution involved few-shot prompting with three example challenges in the prompt, combined with JSON schema validation and retry logic for malformed responses.

5.3.3 Vector Database Selection

Document the selected vector database and the rationale for the choice once finalized. Options under evaluation: pgvector (lowest operational overhead, integrated with PostgreSQL), Qdrant (better query performance at scale), Weaviate (richer filtering).

5.4 Repository and Project Structure

The project is organized as a monorepo with the following top-level directories:

- `/shared` — Shared Kotlin module (KMP).
- `/mobile` — KMP mobile application (`androidApp`, `iosApp`, `commonMain`).
- `/backend` — Spring Boot application.
- `/landing` — TypeScript/React landing page and admin panel.
- `/infra` — Docker Compose files, CI/CD configuration, Kubernetes manifests (planned).

Add repository diagram / package structure once directory layout is finalized.

Chapter 6

Conclusion and Future Work

6.1 Summary of Contributions

This report presents the design and ongoing implementation of Play4Change, an adaptive learning platform powered by generative AI and gamification mechanics. The primary contributions of this project are:

1. A **product concept** that addresses a genuine gap in sustainability and digital literacy education: combining AI-generated content, adaptive delivery, and community engagement in a single, coherent platform aligned with the UN SDGs.
2. A **cross-platform mobile architecture** using Kotlin Multiplatform and Decompose that shares business logic across iOS and Android while preserving native user experience on each platform.
3. A **type-safe system boundary** via a shared Kotlin module, eliminating an entire class of client-server integration bugs at compile time.
4. An **AI content pipeline** (Mistral + RAG + vector search) that reduces the educator's authoring burden from writing every challenge manually to curating AI-generated content.
5. An **engineering architecture** (DDD, hexagonal, clean architecture) that keeps the system maintainable and testable as it grows.
6. A **cost-conscious infrastructure design** with caching, CDN, passwordless authentication, and Kubernetes readiness that ensures the platform can serve a large user base without becoming an operational liability.

6.2 Current Limitations

As the project is still under active development, several limitations apply at the time of writing:

- **No production deployment**: the system is running in a local Docker Compose environment. Production infrastructure has not yet been provisioned.

- **Limited content**: the topic library is populated with a small number of test topics. Educator onboarding workflows are not yet complete.
- **Adaptive algorithm**: the difficulty adaptation algorithm is scaffolded but not yet calibrated. Real learner data is required for tuning.
- **iOS UI incomplete**: the SwiftUI layer for iOS is behind the Android Jetpack Compose implementation due to the complexity of KMP/iOS integration.

6.3 Future Work

The following areas are planned for future development:

6.3.1 Kubernetes Deployment

Migrating from Docker Compose to a Kubernetes cluster will unlock horizontal autoscaling, rolling deployments with zero downtime, and improved resource utilization. Helm charts for the Spring Boot application, PostgreSQL (with persistent volumes), Redis, and the vector database are planned.

Define target cloud provider and cluster size.

6.3.2 CDN and Edge Caching

A production CDN will be configured to serve:

- The React landing page (static assets with long cache TTLs);
- Topic cover images and educator-uploaded media;
- Pre-generated challenge payloads for frequently accessed topics.

6.3.3 Advanced Analytics and Reporting

A learner analytics dashboard will allow educators to track cohort engagement, challenge completion rates, and SDG coverage. Anonymized aggregate data will feed back into the AI pipeline to improve challenge quality over time.

Define data model and privacy constraints for analytics.

6.3.4 Social and Community Features

- **Group challenges**: teams compete on a shared daily challenge, fostering classroom or organizational cohesion.
- **Educator communities**: topic authors can share and remix learning paths.
- **Open topic marketplace**: high-quality topics can be published for any learner to discover, beyond the closed enrollment model.

6.3.5 Richer Adaptive Model

The current adaptive difficulty is based on a simple performance score. Future work includes:

- **Spaced repetition**: schedule challenge reviews based on the learner's forgetting curve (e.g., SM-2 algorithm).
- **Learner interest modeling**: track which SDG tags the learner engages with most and bias content selection accordingly.
- **Multimodal challenges**: integrate audio or image-based challenges for learners who respond better to non-textual content.

6.3.6 Accessibility Enhancements

Audit WCAG 2.1 compliance for both mobile and web surfaces. Define accessibility roadmap.

6.3.7 Internationalization

The platform currently supports English and Portuguese. Adding multilingual content generation (Mistral supports multiple languages) and UI localization will open the platform to a global audience aligned with the SDGs' international scope.

Define target languages for initial i18n rollout.

6.4 Conclusion

Play4Change demonstrates that it is possible to build an educational platform that is simultaneously engaging, rigorous, cost-sustainable, and architecturally sound. The combination of Kotlin Multiplatform, a shared type-safe module, DDD/hexagonal/clean architecture, and an AI content pipeline represents a coherent answer to the principal challenges of edtech: content bottlenecks, low engagement, high infrastructure cost, and poor personalization.

The project is on track to deliver a working end-to-end system by the end of the 2025/26 academic year. The foundation — architecture, shared module, core backend, and mobile skeleton — is in place. What remains is the completion of the remaining use cases, the AI pipeline calibration, and the production deployment. Each open item has been annotated in this document with a `\todo` note to guide the final stages of development.

The author believes Play4Change has the potential to make a meaningful contribution to sustainability education and digital literacy — not just as a student project, but as a platform that could operate at scale and create real behavioral change.

Appendix A

Architecture Diagrams

A.1 System Component Diagram

Insert full system component diagram (C4 Level 2 or 3) once finalized.

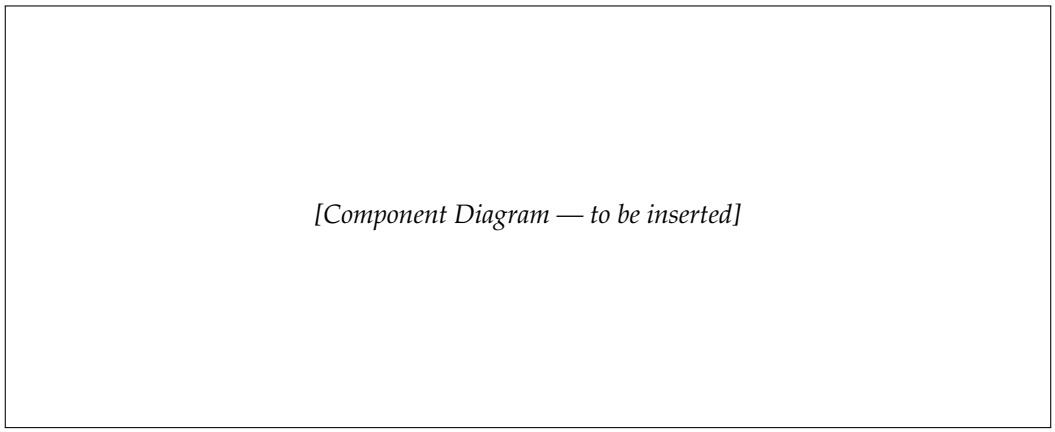

Figure A.1: Play4Change system component diagram.

A.2 Domain Model

Insert aggregates and value objects diagram per bounded context once finalized.

A.3 AI Pipeline Sequence Diagram

Insert sequence diagram for content ingestion and challenge generation once finalized.

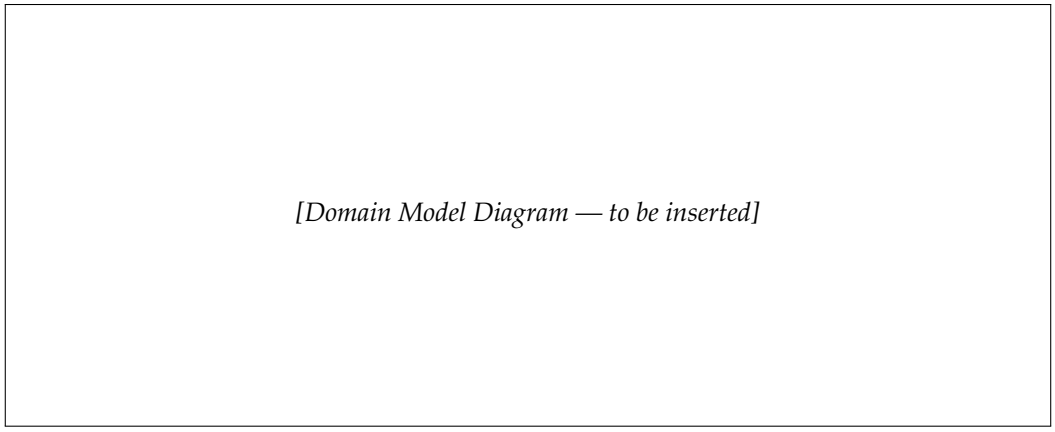

Figure A.2: Play4Change domain model overview.

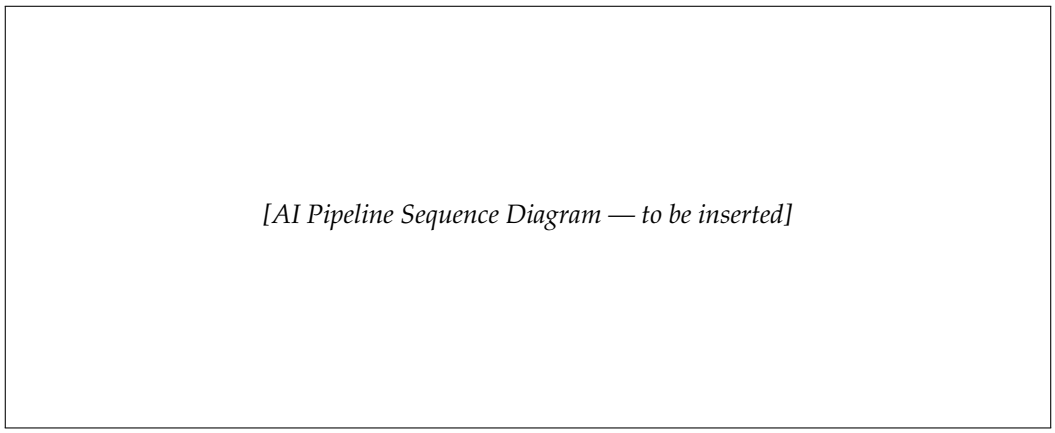

Figure A.3: AI content pipeline: ingestion, embedding, retrieval, and generation.

A.4 Database Schema (ERD)

Insert Entity-Relationship Diagram for the relational schema once finalized.

A.5 API Endpoint Reference

Link to or embed the OpenAPI (Swagger) specification for the Play4Change REST API once generated.

A.6 Technology Stack Summary

Table A.1: Complete technology stack summary.

Layer	Technology	Notes
Mobile Client (shared)	Kotlin Multiplatform	Business logic, API clients
Mobile Client (Android)	Jetpack Compose	Native UI
Mobile Client (iOS)	SwiftUI	Native UI
Navigation	Decompose	Lifecycle-aware, shared
Local DB (mobile)	SQLDelight	Type-safe, KMP
HTTP Client	Ktor	KMP
Serialization	Kotlinx.serialization	Shared module
Backend	Spring Boot 3 / Kotlin	JVM 21
Database	PostgreSQL	Primary data store
Cache	Redis	Multi-level caching
Vector Store	TBD (pgvector / Qdrant)	RAG embeddings
AI Model	Mistral	LLM for content generation
Containerization	Docker / Compose	Dev and staging
Orchestration (planned)	Kubernetes	Production
Web Frontend	TypeScript + React	Landing page, admin panel
CI/CD	TBD (GitHub Actions)	Lint, test, build, deploy
Auth	Passwordless (magic link)	No password hashes